Pre-submission validation research

Observation-Guided Combination Testing for Coding-Agent Patches

Program analysis + bounded semantic reasoning

16/17 development trials detected

Unified abstraction still under development

Research progress report

30 September 2026

Score: completed fresh v3 run; 14 distinct tasks. Detection is not issue resolution.

The problem: a patch passes its validations but remains wrong

Available before submission

- Issue and candidate patch.
- Repository and full original trajectory.
- Tests, scripts, failures and fixes.

Replay the historical state to simulate a plugin before submission.

Effective validation gap

Validations exercise individual behaviors but miss an interaction, or use an unsuitable observation or expectation.

Goal: add or modify a valid combination test that exposes the submitted patch's defect.

Intended endpoint: test feedback → patch repair → hidden-test evaluation. This repair loop is not yet evaluated.

Setting: DeepSWE replay; original trajectory is allowed input. Hidden tests and reference patches are evaluation-only.

Real DeepSWE failure: HTTPX streaming JSON (N01)

Issue: incremental JSON iteration must reject malformed data and close a consumed streaming response.

Validation context	Valid JSON	Invalid JSON
In-memory response	Tested	Tested
Live response stream	Tested	Missing close check

Normal streaming tests checked consumed/closed state; malformed-line tests used in-memory content.

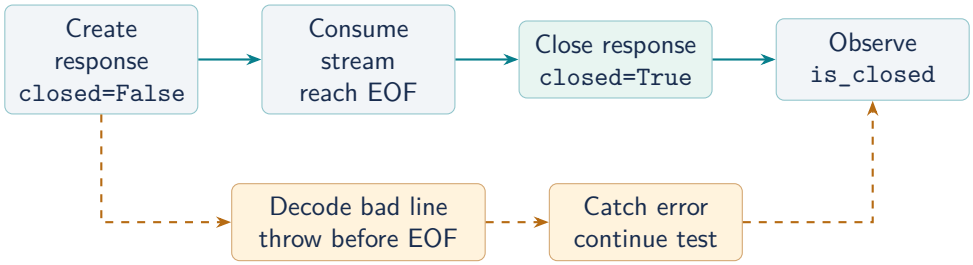
What escaped validation?

A complete malformed NDJSON line is decoded **before the stream reaches EOF**. Parsing throws; normal close is skipped.

Original agent: `deepseek-v4-flash`. The trajectory records **156 passing tests** after incremental parsing was introduced.

Public N01 trial; trajectory steps 57, 77–82, 95. Matrix summarizes relevant checks, not all tests.

N01: the error must occur before the write that closes the stream



The same response and the observation after the exception matter. A malformed buffered JSON response may already be closed and reveal no defect.

N01 candidate behavior and original trajectory. Arrows illustrate execution order; they do not claim a complete static proof.

Four challenges determine whether a combination is useful

Find the useful validation

Many temporary tests and variants compete for a small review budget. Historical tests may also become stale after later edits.

Make interaction observable

Inputs must change what the assertion can see. One-field schemas or normal completion can hide the interaction.

Connect the right context

Operations must concern the right object, input source and observation stage. Co-occurrence alone is insufficient.

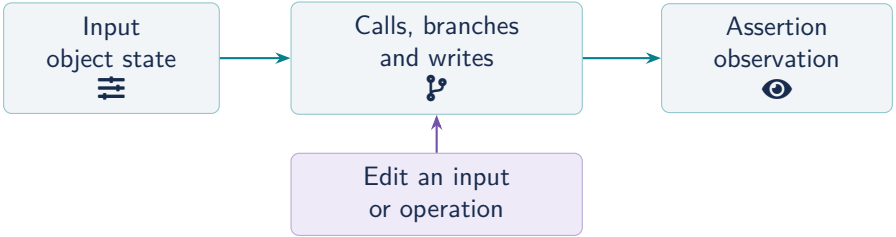
Keep a correct oracle

A combined context can invalidate old expectations. The submitted implementation may be wrong, so its output is not ground truth.

Observed across 17 development trials: N01 exception timing; N09 partial defaults; N05 oracle interpretation; N06 historical validation recovery.

Core abstraction: the production path of an observation

Unit: an assertion on a specific object at a specific execution stage, together with the operations producing its observed value.



Search question
Can an edit change how this state is produced, while keeping the observation reachable?

Semantic question
What should hold in the new context, according to the issue and API contract?

Research abstraction; bounded pieces are implemented. It is not a complete dependence model or a unified composition algorithm.

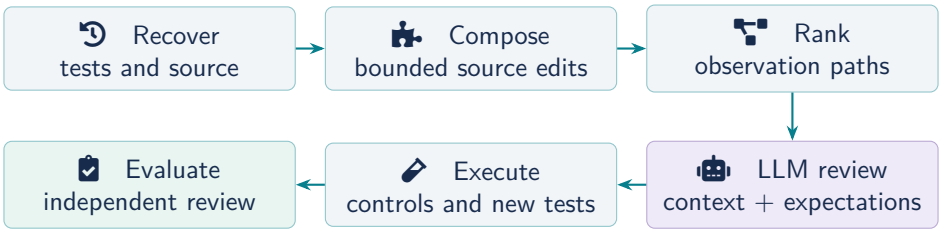
Why an observation path helps expose a missing interaction

Executable context	Visible interaction	Contract reasoning
Reuse source inputs and calls; align their arguments and receiver objects.	Trace the operations producing the observed value, including its stage.	Provide relevant source facts; decide whether every expectation still holds.
N01: malformed bytes enter the same live response whose close state is checked.	**N09:** read the omitted default fields in the pending record, before flush.	**Both:** existing close/default expectations remain correct and expose the defect.

Analysis organizes the relevant program facts; the model reasons about a concrete combination and its complete expected behavior.

This explains the mechanism in two confirmed examples. It is not a completeness or generalization guarantee.

Overall workflow: source structure narrows semantic reasoning



Iteration: preserve unserved candidates; expand the source frontier.

Next: use valid failures for patch repair and hidden-test evaluation.

Scored v3 uses existing writer/impact scheduling. The newer patch-to-observation ranking is a separate optional prototype.

How existing tests become executable combination candidates

Parse tests and assertion APIs from repository and trajectory artifacts. Preserve their definitions, calls, fixtures and imports.

Source-derived building blocks

- Inputs and argument positions.
- Calls, bindings and object reuse.
- Assertions and required execution prefixes.

Finite edits: parameter replacement, operation composition, shared-object binding and field-presence changes.

N01: reuse two existing contexts

Streaming test:
live response + close observation

Error test:
malformed NDJSON + decoding-error check

Combination:
malformed input in the live response; observe close after catching the error.

Executability does not establish semantic correctness. Deriving new edits directly from paths remains open.

Latest ranking prototype: distinguish edits inside the same test

```
# Submitted patch changed this return.
def calculate(value, unused):
    return value * 3

result = calculate(1, 10)
assert result == 3
```

Candidate A: change 1 to 2.
Candidate B: change 10 to 11.

Candidate-conditioned dependency

Both candidates call the modified function. Only A's edited argument participates in the return value seen by the assertion.

Strict: project arguments through finite return dependencies.

May: use a conservative same-call path to a modified callee; may include unused arguments.

Reparse the edited test and join at the same call site. A missing path remains unknown.

Illustrative fixture; not a DeepSWE result. The optional ranking prototype's full comparison is pending.

The LLM gets one concrete semantic task

Program-produced evidence

- Fixed input/operation edits.
- Complete test and all assertions.
- Calls, arguments and source bodies.
- Issue contract and analysis unknowns.

Semantic decision

Is this a legal context? Which expectations remain valid? What should replace the expectations that change?

Output: retain, revise, or unknown. Code applies supported edits to expectations, preserving actions and observations.

Infer expectations from the contract, not the buggy output. Review all assertions and prerequisites.

Completed v3: at most 16 semantic candidates per trial per round; one full candidate per request; fresh DeepSeek-v4-flash requests.

Formal problem: find a valid defect-exposing combined context

Input: candidate program P, issue contract I, repository R, original trajectory H, and budgets B.

$$T_0 = \mathsf{Extract}(R, H), \qquad \mathcal{C} = \{\mathsf{Apply}(t, \delta) \mid t \in T_0,\ \delta \in \mathcal{E}(T_0, R)\}.$$

T_0: recovered tests; $\mathcal{E}$: finite input/operation edits; $\mathcal{C}$: combined contexts before oracle review.

Search objective

Find $c \in \mathcal{C}$ and reviewed expectations o. Materialize $u = \mathsf{Test}(c, o)$ such that:

$$\mathsf{Valid}_I(u) \ \wedge \bigwedge_{s \in \mathsf{Controls}(u)} \mathsf{Pass}(P, s) \ \wedge \ \mathsf{IssueFail}_I(P, u).$$

Valid: legal test and correct complete expectations. **Controls**: associated source tests. **IssueFail**: confirmed issue-related failure, including a defect-induced panic.

Independent semantic review; compile/environment errors excluded. Budgets limit composition, requests and execution.

Formal success criterion: a valid, issue-related native failure

$\mathrm{Valid}_I(u)$: semantic validity

Legal setup, meaningful observations and correct complete expectations, including assertions before the final observation.

$\mathrm{Pass}(P, s)$: source control

Associated original tests pass on the same replayed candidate program.

$\mathrm{IssueFail}_I(P, u)$: defect evidence

Confirmed issue-related native test failure: an assertion mismatch or a defect-induced exception/panic under valid setup.

Compile and environment errors do not qualify. Semantic validity is assessed independently; our analyzer does not prove it.

The defect escaped effective pre-submission validation. Its combined relation need not be absent from every earlier trajectory step.

N06 recovers a historical combination not revalidated on the final patch. P5 exposes a defect-induced VM panic before later assertions.

Formal abstraction: edits, production paths and observations

For an edited test u, construct a bounded dependence graph $G_u = (V_u, E_u)$ from the actual edited source.

Computable components

- D_u: input/operation edit nodes.
- Δ: modified production code.
- a_q: actual-value node of assertion q.
- Edges: value, control, supported calls/effects.

Path-relevance query

$$d \in D_u \rightsquigarrow p \in \Delta \rightsquigarrow a_q$$

$\rightsquigarrow$: supported dependence reachability. Join compatible bindings at the **same statically identified call site**.

N01: malformed bytes affect decoding; an exception bypasses the observed close write.

Limit: a path helps reasoning; it does not prove feasibility, oracle correctness or complete coverage.

Bounded research definition; heap/exception modeling is incomplete and differs across implemented modules.

Formal method: stable path priority, bounded oracle review

Let F be the deterministic family/seed/shape order of the fixed candidate pool.

Two predeclared path predicates

$S(u)$: an edited input reaches modified code through supported return dependencies and then the actual observation.

$M(u)$: an edited input affects a concrete call in the actual-value slice; its reliable callee can reach modified code. This is weaker may evidence.

$$\mathrm{Rank}_m(F) = [u \in F \mid m(u)] \;\|\; [u \in F \mid \neg m(u)], \quad m \in \{S, M\}.$$

$\|$: concatenation; partitions preserve F's order. Fallback includes unsupported and unknown candidates. **Review:** select up to B_o requestable, unserved candidates per round; infer expectations and execute. B_o is the semantic budget. Keep pending candidates.

Latest optional strict/may ranking and CLI. The frozen 16/17 run used earlier writer/impact scheduling; no result is attributed to this new rule.

N01 worked example: the combined test checks close after failure

```
def chunks():
    yield b'{"a": 1}\nnot json\n'
    yield b'1}'  # unread when parsing fails

response = httpx.Response(
    200,
    headers={"Content-Type":
             "application/x-ndjson"},
    stream=IteratorByteStream(chunks()),
)
with pytest.raises(httpx.DecodingError):
    list(response.iter_json())
assert response.is_closed
```

Executed witness condensed: imports, names and additional checks omitted here.

Why this input works

A malformed complete line is parsed while the stream is live. Catch the error, then observe close.

Contract: close after streaming consumption.

Required: `True`.
Candidate returns: `False`.

The original close oracle is retained.

Executed N01 witness c8e075c82686eb21c4222575; source and native execution in the v3 ledger.

N01 worked example: evidence from recovery to native failure

Stage	What actually happened in the completed v3 run
Recover	Stream construction and close checks; malformed donor input and error handling.
Compose	Align the response; transfer input, header and exception context into its live generator.
Select/review	Exception/write context informs priority; retain the original close oracle.
Execute	Source controls pass. Generated pytest test fails: `is_closed` is `False`.
Confirm	Independent evaluation checks the complete test, emitted source and native failure.

A complete native defect witness on the replayed submitted patch; the patch itself is not repaired.

N01 deliveries and runtime index; main v3 `checkpoint_14.json`.

Second real success: Koota partial defaults before flush (N09)

Contract: deferred `get` must return the same logical values as after `flush`. A partial parameter object must retain omitted defaults.

```
// Existing Test fixture defaults include:
// current=1, test='hello', bool=true.
const entity = world.spawn();
world.deferred.add(entity,
  Test({current: 2, arr: ['d','e','f']}));
const test = entity.get(Test); // no flush
expect(test).toMatchObject({
  current: 2, test: 'hello', bool: true,
  arr: ['d','e','f']
});
```

Condensed executed witness; the complete original fixture, class/default checks and later actions are preserved in the actual test.

Recover: repository test for ordinary partial `entity.add`.

Compose: replace that call with `world.deferred.add`; preserve bindings and expectations.

Observe: read before flush. The buggy projection returns supplied parameters without omitted defaults.

✅ Source control passes; generated Vitest test fails on missing defaults.

Public N09 trial; main witness `08f2c625387570b30312`. Source expectations retained.

Evaluation protocol: a confirmed witness, not any failing test

Completed main experiment

- 17 trials, 14 distinct tasks.
- 7 original pilots + 10 Flash failures.
- All used during development.
- Frozen v3: `7a8ff95`; 2 rounds.

Fresh oracle requests; original trajectories available. No prior-answer reuse.

Trial-level detection metric

Count a trial only with legal context, correct complete expectations, passing controls and a named issue-related failure.

Separate metrics: target exposure/rank, complete oracle correctness, execution errors, calls and tokens.

Rank, test correctness and issue resolution are different metrics. Overall false-positive rate is not yet established.

v3 independent evaluation, `checkpoint_14.json`. P1–P3 share a task; P5/P6 share another.

All 17 trials: 16 confirmed witnesses, one remaining miss

Original seven pilots

Case	Interaction	Hit	Oracle
P1	Textual: event phase	✓	Revised
P2	Textual: shortcut + text	✓	Retained
P3	Textual: shifted alias	✓	Retained
P4	IPython: quoted path	✓	Retained
P5	Tengo: returned callable	✓	Retained
P6	Tengo: shared cells	✓	Revised
P7	Valibot: recursive transform	✓	Revised

Pilots have several original agent models. The ten added trials all use Flash.

Ten DeepSeek-v4-flash trials

Case	Interaction	Hit	Oracle
N01	HTTPX: JSON error + close	✓	Retained
N02	HTTPX: multipart + close	✓	Retained
N03	mnamer: duplicate + quota	✓	Retained
N04	sqlite-utils: bulk + invariant	✓	Revised
N05	termenv: fitting + reset	!	–
N06	etree: sibling insert + text	✓	Retained
N07	go-git: overlapping edits	✓	Retained
N08	Pebble: exclusive frontier	✓	Retained
N09	Koota: partial defaults	✓	Retained
N10	DDBToolbox: omitted mode	✓	Retained

N06 recovers a historical relation; N07 uses an alternative witness; N08 uses new valid premises. DDBToolbox = DynamoDB Toolbox.

12 retained source oracles + 4 revised = 16 detected. All are development trials, not held-out generalization.

Results: practical progress with limited causal evidence

Completed main run

16/17

Confirmed development-trial witnesses



540 fresh calls; 7.42M total tokens.
83 responses reached the output limit.

Isolated local evidence

Same eight Python pools, eight proposals each, no LLM: writer-path static priority improved confirmed detection **3/8** → **4/8**.

Added N01. This supports a local search rule, not a universal abstraction.

Earlier 12/17 and current 16/17 use different configurations; their difference does not isolate the analysis contribution.

Main score: v3 checkpoint 14 and provider receipts. Local equal-budget experiment: assertion-writer search report. Tokens include reasoning once.

Academic novelty: a contribution hypothesis with a clear boundary

Candidate insight: search for edits affecting one observation's production path; ask the model only about the remaining contract semantics.

Design element	Current academic assessment
Candidate-specific query	Joins edit, patch and observation at one call site. Its precision benefit is unverified.
Classical analysis	Slicing, binding, return summaries and call graphs are established.
Analysis + LLM	Precedent in STARS; this division alone is not novel.
Missing capability	A unified path-driven synthesis operator: paths currently rank existing candidates.

Hypothesis to test: observation-driven search improves valid defect detection at equal budget.

Not a demonstrated unified innovation. STARS, ChaCo and Testora supply relevant baselines; see references.

Current limitations: selection, oracle reliability and analysis cost

N05: an unresolved complete witness

Fitting width + preserve reset. Target ranks **96 and 122** in the two rounds; never offered. A related executed variant kept an invalid truncation expectation.

Both budget allocation and complete oracle inference remain bottlenecks.

Source-analysis prototype is costly

A 370-candidate Go pool took **33.2 minutes** in strict mode, also computing may evidence. This is whole-pool cost.

Helper, heap, callback and scope support is incomplete. The new 17-trial ranking census is still pending.

34 sampled raw failures: 30 invalid tests, 2 valid detections, 2 uncertain contracts. This is not the overall false-positive rate.

N05 final audit; independent failure sample; patch-priority v1 census, P5 strict = 1990.317 seconds. No new ranking/detection gain is claimed.

Next research steps: make the abstraction drive synthesis

Path-driven composition

Derive new edits from paths, beyond current constructors. Compare under equal budgets.

Bounded analysis cost

Index assertions and cache call facts. Measure whole-pool and online latency.

Complete observations

Preserve object identity, phases and prerequisites. Audit every assertion against contract evidence.

Independent evaluation

Freeze unseen trials; repeat model runs; evaluate repair with hidden tests.

Success: more valid witnesses and better repaired patches at fixed total budget, with measured misses and false alarms.

Planned work. Current evidence establishes local detection progress, not zero false negatives, generality, or issue-resolution improvement.

Reference points for the research design

- **STARS, ISSTA 2026**: static analysis synthesizes assertion components, including type-connected observer call chains; LLMs concretize assertions.
- **Change And Cover, ICSE 2026**: patch-focused regression-test augmentation and recovery of project-specific test context; assumes a bug-free PR.
- **Testora, ICSE 2026**: compares pre/post-change behavior and uses documented change intent to classify differences.
- **Mixed-Assertion Scenario, ISSTA 2026**: multiple assertions jointly express test objectives; known insertion positions are a consequential assumption.

Our comparison must isolate candidate composition, observation relevance and oracle correctness. Classical analysis and context retrieval are strong baselines.

Primary author papers and official conference abstract. These comparisons motivate design choices; they do not establish global novelty.

Evidence map and reproducible scope

Material	Repository-relative location
Current method	`research/method_design.md`
Original trials	`research/expanded_trials_20260926/cases.json`; `data/relational_gap_v1/N01/trajectory.json`; N09 likewise
Main score	`results/relation_search_20260929_v3/evaluation/uniform_fresh_impact_v3/checkpoint_14.json`
Native witnesses	Same directory: N01 deliveries and N09 runtime index
Implementation	`relation_search/`; `scripts/run_relation_campaign.py`
New ranking	`experimental/patch_observation_priority.py`
This presentation	`docs/combination_testing_research.tex`

Frozen distinction: scored v3 is `7a8ff95`; later source-priority integration is `7615f4e`. New prototype results do not overwrite the scored run.

Report snapshot: 30 September 2026. Public issue/trajectory/patch evidence; no hidden verifier or reference patch is used for generation.